

PCA and Data Visualization: A Study of Model Reduction, Digit Recognition, and Iris Classification

Mojdeh Fatehpour and Mohammadmatin Rezanezhadchoulai

Università degli Studi della Campania "Luigi Vanvitelli"

Professor Rosanna Campagna

Abstract:

In this report, we apply Principal Component Analysis (PCA) to three different datasets to understand how it can help reduce dimensionality and reveal important patterns. The first dataset is synthetic, with 4000 points in 6 dimensions. We used scatter plots and singular value analysis to check if the data could be represented in a lower-dimensional space. The second dataset contains handwritten digit images represented as 256-dimensional vectors. We selected a few digits and reconstructed them using different numbers of principal components to see how well PCA can compress the data and still keep the important features. In the third dataset, which includes flower measurements from three Iris species, we used PCA to check if the different species can be separated based on their features.

Overall, the results show that PCA is useful for both compressing data and making patterns easier to see. In each case, we used proper centering, applied SVD, and evaluated the results with plots and residual errors. This helped us understand how much information is kept with fewer components and whether clusters or separations between groups can be seen more clearly in lower dimensions.

1. Exercise 1: PCA on 6-Dimensional Synthetic Data

1.1. Problem Description

Download from Blackboard the file `ModelReductionData.mat`. The file contains a data matrix $X \in \mathbb{R}^{6 \times 4000}$. Each column of X represents a data vector in six-dimensional space.

- (a) Visualize the raw data using scatter plots: select two components at a time and plot them against each other. There are $\binom{6}{2} = 15$ two-dimensional projections. Suggested MATLAB code:

```
1 load ModelReductionData
2 figure(1)
3 plot(X(1,:), X(2,:), 'k.', 'MarkerSize', 7)
4 axis equal
5 set(gca, 'FontSize', 20)
```

- (b) Center the data and compute the SVD. Plot the singular values. What can you say about the dimensionality of the data?
- (c) Show scatter plots of the first few principal components. Do the plots suggest a presence of clusters?

1.2. Methodology

The dataset `ModelReductionData.mat` contains a matrix $X \in \mathbb{R}^{6 \times 4000}$, where each column is a data point in six-dimensional space. Our goal is to understand the data structure by visualizing relationships between its components and identifying its intrinsic dimensionality using PCA.

- **Raw Data Visualization:** We began by plotting all pairwise 2D projections of the six components, resulting in $\binom{6}{2} = 15$ scatter plots. This helps to visually assess whether clusters or patterns are present in the original high-dimensional data.
- **Centering the Data:** Before applying PCA, we centered the data by subtracting the mean of each row. This step is essential because PCA assumes the data is centered at the origin; otherwise, the first principal component might align with the mean instead of the direction of maximum variance.
- **Singular Value Decomposition (SVD):** We computed the SVD of the centered matrix X_c to obtain the principal directions and singular values. The singular values represent the amount of variance explained by each principal component. By plotting them, we can infer the effective dimensionality of the dataset.
- **Principal Component Visualization:** Finally, we projected the data onto the first three principal components and created scatter plots of PC1 vs PC2, PC1 vs PC3, and PC2 vs PC3. These plots help us determine if the lower-dimensional representation reveals underlying clusters or structure not easily visible in the original space.

```

1 xbar = mean(X, 2);
2 Xc = X - xbar;
3 [U, S, V] = svd(Xc, 'econ');

```

1.3. Results and Figures

Raw Data: 2D Scatter Plots (15 Projections)

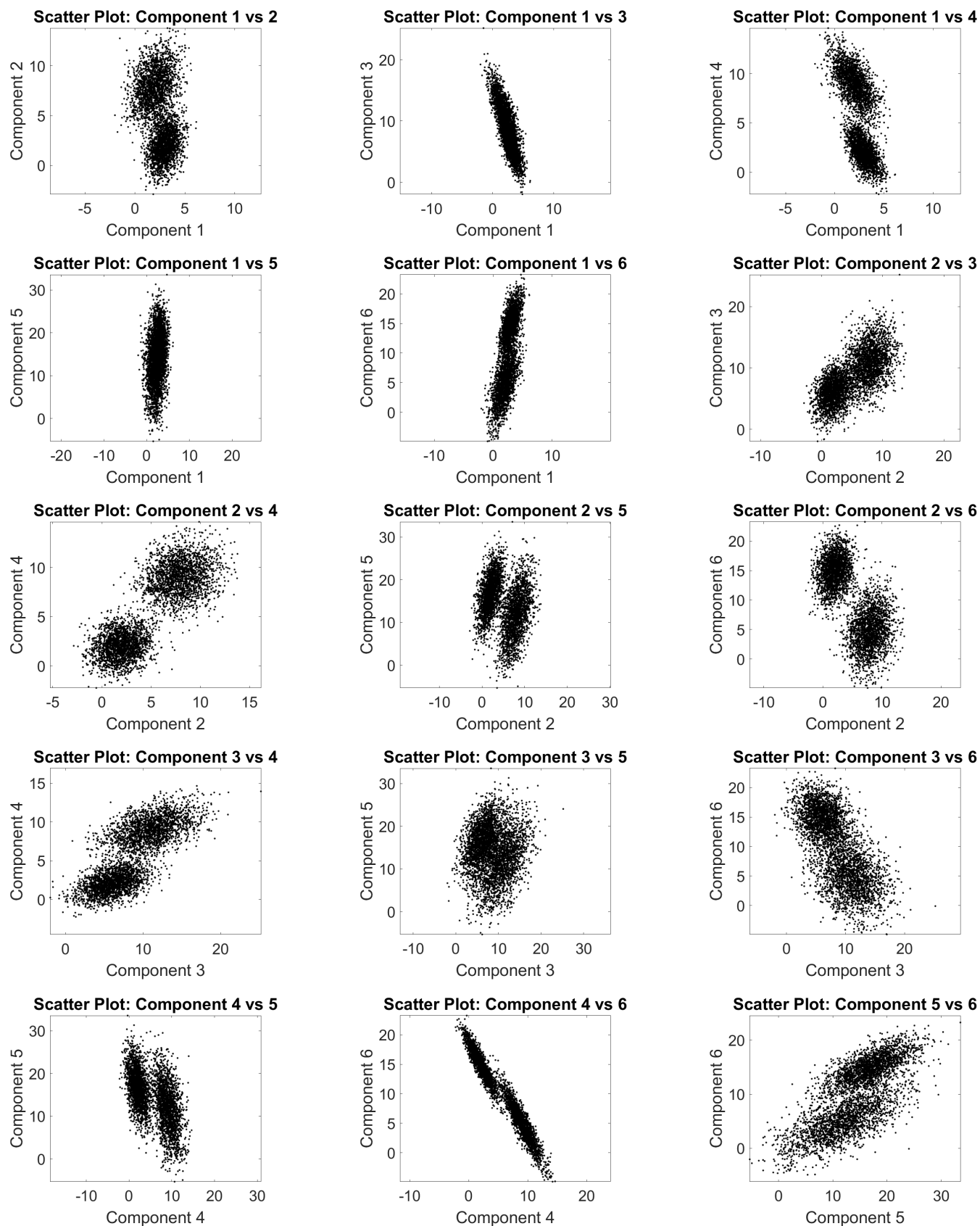


Figure 1. All 15 scatter plots of raw data projections.

Explanation: The scatter plots provide an initial visual exploration of the dataset's structure. Among the 15 pairwise projections, several—including (Component 2 vs 4), (2 vs 6), and (4 vs 6)—show noticeable cluster separation, while others appear more overlapping or elongated. This suggests that the data may not be evenly spread across all six dimensions but could instead lie on a lower-dimensional subspace. These patterns motivate the use of PCA to extract meaningful directions of variance and to better visualize potential groupings that are otherwise hidden in the original high-dimensional space.

Singular Value Analysis

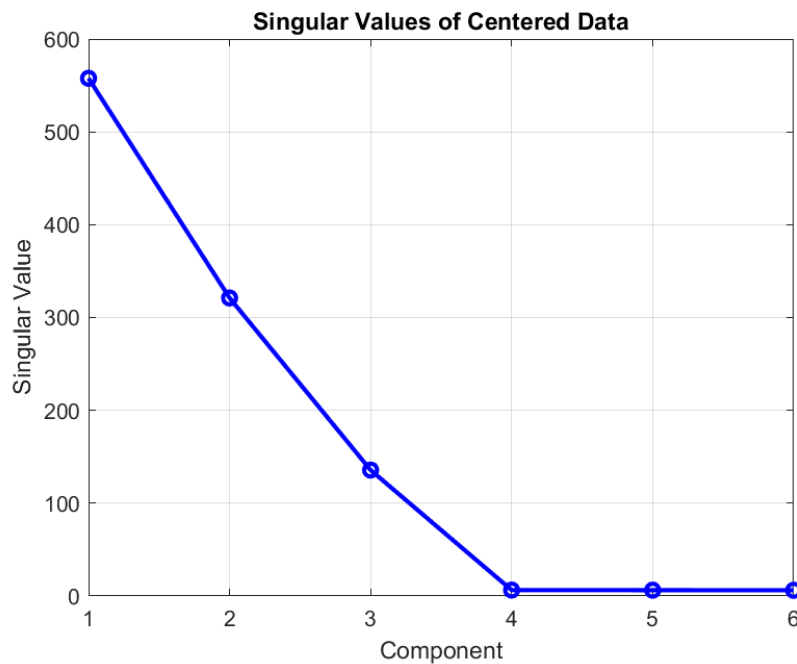


Figure 2. Singular values of the centered data matrix.

Interpretation: The plot of the singular values shows a steep drop after the third component, with the last three values being close to zero. This indicates that most of the variance in the data is captured by the first three principal components. From this, we can conclude that the dataset is effectively three-dimensional, even though it originally lies in a six-dimensional space. This dimensionality reduction justifies using a 3D PCA projection for further analysis and visualization, as very little information is lost by discarding the lower singular values.

PCA Projections

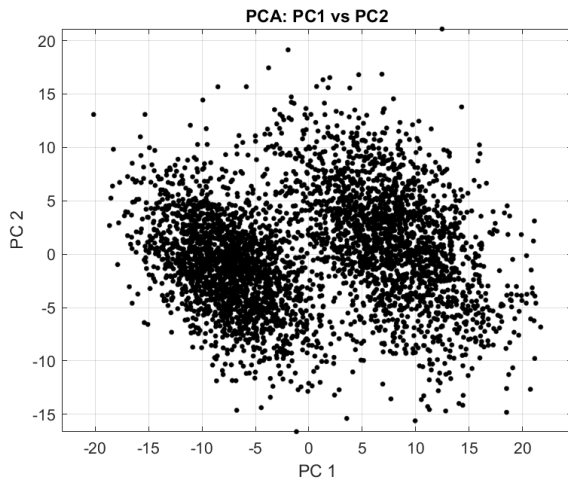


Figure 3. PC1 vs PC2 projection.

Explanation: The scatter plot of the first two principal components (PC1 vs PC2) reveals two visibly dense regions, suggesting the presence of distinct clusters in the data. This is important because the original 6D data did not clearly show such separation. PCA has successfully projected the high-dimensional structure into 2D while preserving essential patterns. This visualization highlights that much of the underlying group structure can already be observed using just the first two principal components.

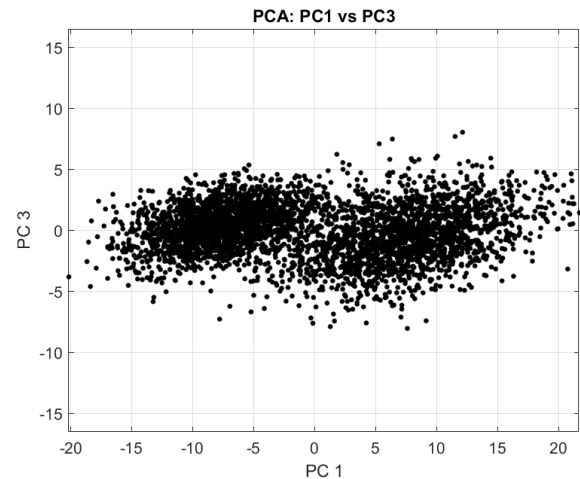


Figure 4. PC1 vs PC3 projection.

Explanation: The PC1 vs PC3 projection shows that the spread along PC3 is narrower compared to PC1, meaning PC3 captures less variance. However, the slight vertical dispersion along PC3 indicates subtle internal structure—possibly sub-clusters—within the broader groups identified by PC1. While PC3 alone does not reveal distinct clusters, it adds a third dimension that complements PC1 and helps uncover finer details in the data.

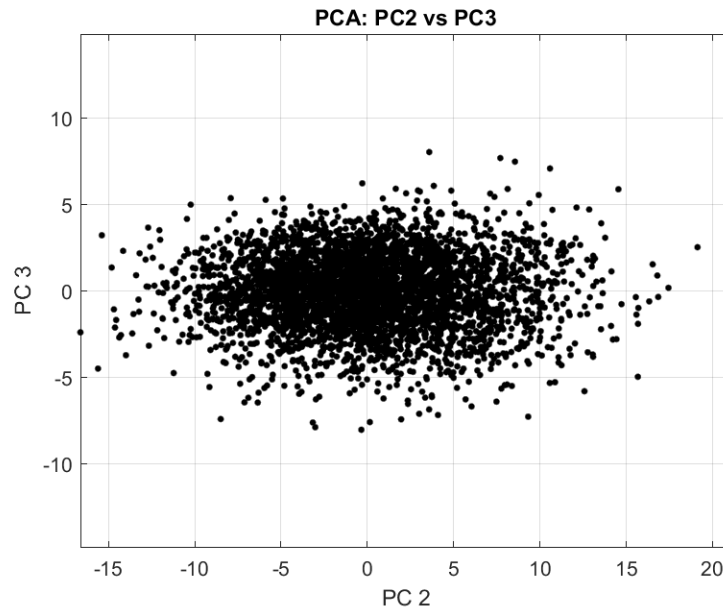


Figure 5. PC2 vs PC3 projection.

Explanation: The projection on PC2 vs PC3 shows a tight concentration of points along PC3, with minimal vertical spread. This suggests that PC3 contributes relatively little variance compared to PC1 and PC2. Although this view does not reveal distinct clusters on its own, it complements the other PCA plots by reinforcing the idea that most of the structure is captured by the first two components. The low variance in PC3 further supports dimensionality reduction to 2D or 3D.

2. Exercise 2: Image Compression Using PCA

2.1. Problem Description

Download from Blackboard the data file `HandwrittenDigits.mat`, containing the data matrix X of size 256×1707 containing pixel images of the handwritten digits, and the label vector I of length 1707 containing numbers from 0 to 9, indicating the digits that the corresponding images represent.

Extract from X the images that correspond to numbers 0, 1, 3, and 7. From each subgroup, select 5 samples, and approximate these samples by the linear combination of the first k feature vectors, $k = 5, 10, 15, 20, 25$, that is,

$$\hat{x}^{(j)} = \sum_{i=1}^k z_i^{(j)} u_i$$

where $z^{(j)}$ are the principal components of $x^{(j)}$. Plot the approximation as images, as well as the residuals, $x^{(j)} - \hat{x}^{(j)}$. Plot the norms of the errors as a function of k .

2.2. Methodology

The dataset `HandwrittenDigits.mat` includes a matrix $X \in \mathbb{R}^{256 \times 1707}$, where each column represents a 16×16 grayscale image of a handwritten digit, and a vector I of labels indicating the digit (from 0 to 9) for each column.

Our objective in this task was to study PCA as a tool for image compression and noise reduction. We focused on four specific digits — 0, 1, 3, and 7 — and evaluated how well a small number of principal components could reconstruct the original images.

- **Data Selection:** We first selected 5 images for each of the four digits (0, 1, 3, 7), resulting in a total of 20 images. This subset was stored in a new matrix, $X_{\text{sel}} \in \mathbb{R}^{256 \times 20}$.
- **Centering the Data:** The dataset was centered by subtracting the mean image vector across all 20 columns. Centering ensures that the principal components capture directions of maximum variance, and not just the overall brightness of the images.
- **PCA via SVD:** We applied Singular Value Decomposition (SVD) to the centered data matrix:

$$X_c = UDV^T$$

The matrix U contains the principal directions (eigenfaces), and the decomposition allows reconstruction of images using only a subset of these components.

- **Image Reconstruction:** Each of the 20 selected images was approximated using the top k principal components, with $k = 5, 10, 15, 20, 25$. For each value of k , the reconstruction was computed as:

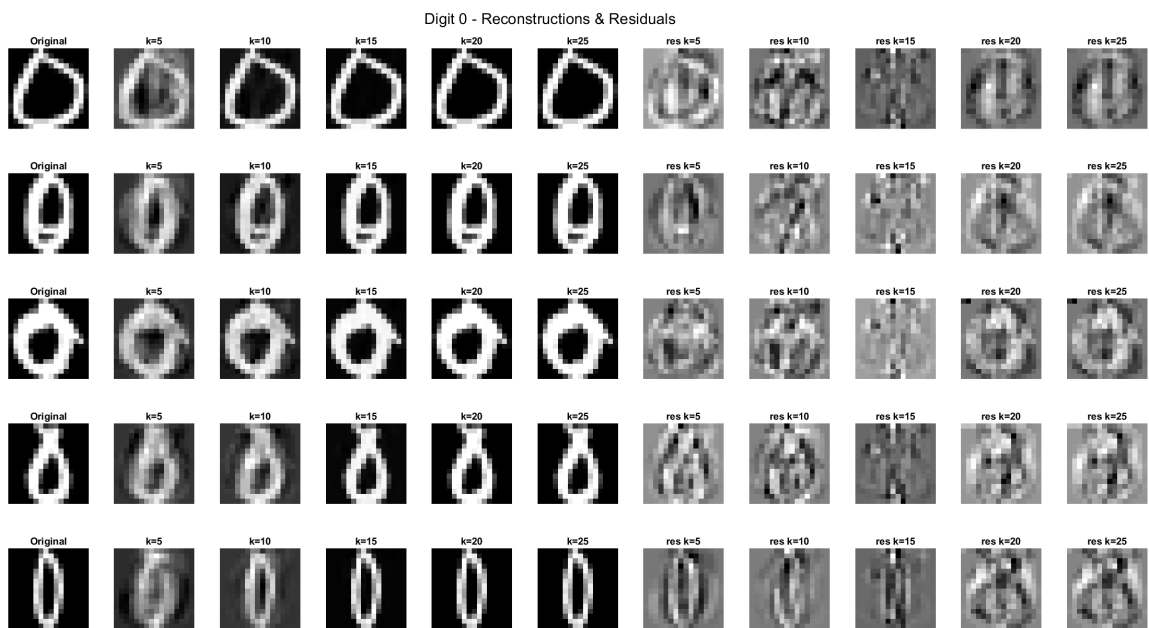
$$\hat{x}^{(j)} = \sum_{i=1}^k z_i^{(j)} u_i$$

where $z_i^{(j)}$ are the projection coefficients (scores) and u_i are the first k columns of U . The reconstructions were then compared to the original images.

- **Residual and Error Analysis:** For each reconstruction, we computed the residual image by subtracting the approximation from the original, and visualized the residuals. To quantify the reconstruction quality, we calculated the error norm $\|x - \hat{x}\|$ for each k , and plotted it to observe how error decreases with increasing number of components.
- **Visualization:** Each digit was presented in its own figure. For every sample, we included the original image, all five reconstructed versions, and the corresponding residuals. Additionally, we plotted the error norm curves for all 5 samples of each digit in a single graph for comparison.

```

1 Uk = U(:, 1:k);
2 z = Uk' * Xc(:, col);
3 xk = Uk * z + xbar;
4 residual = X_sel(:, col) - xk;
```

Code 1. Reconstruction for given k **2.3. Figures and Observations****Figure 6.** Digit 0: Reconstructions and residuals for varying k values.

Explanation: The reconstructed images of digit 0 become progressively clearer as the number of principal components k increases. At low values of k (e.g., $k = 5$), the general shape of the digit is visible but lacks sharpness and detail. As k increases to 10, 15, and 20, the reconstructions gain finer structural elements and increasingly resemble the original digit. By $k = 25$, the images are almost indistinguishable from the originals.

The residuals (difference between original and reconstructed images) provide further insight. With low k , the residuals are rich in structure and intensity, indicating that important features are missing from the reconstruction. As k increases, the residuals fade, showing that the reconstruction error is decreasing and most of the relevant information is captured by the leading principal components.

This pattern confirms that PCA is effective for compressing image data while preserving visual integrity. For digit 0, a relatively small number of components already delivers a good approximation, highlighting the low intrinsic dimensionality of the image structure.

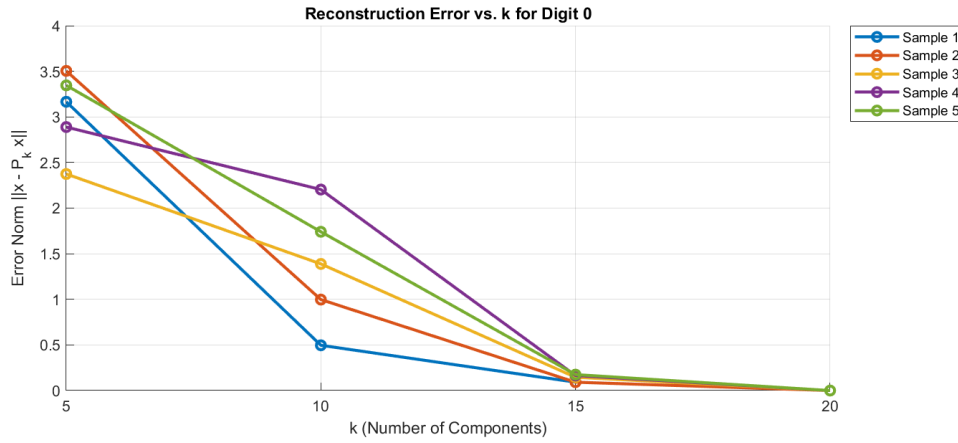


Figure 7. Reconstruction error norm vs. k for Digit 0.

Explanation: The reconstruction error, measured by the norm $\|x - P_k x\|$, decreases steadily as the number of principal components k increases. This indicates that higher values of k capture more of the image's structure and detail. Across all five samples of digit 0, the error reduces sharply between $k = 5$ and $k = 15$, becoming almost negligible by $k = 20$. This suggests that around 15–20 components are sufficient for high-quality approximation, and further increasing k leads to minimal additional benefit. The plot illustrates how PCA can effectively compress image data while preserving accuracy.

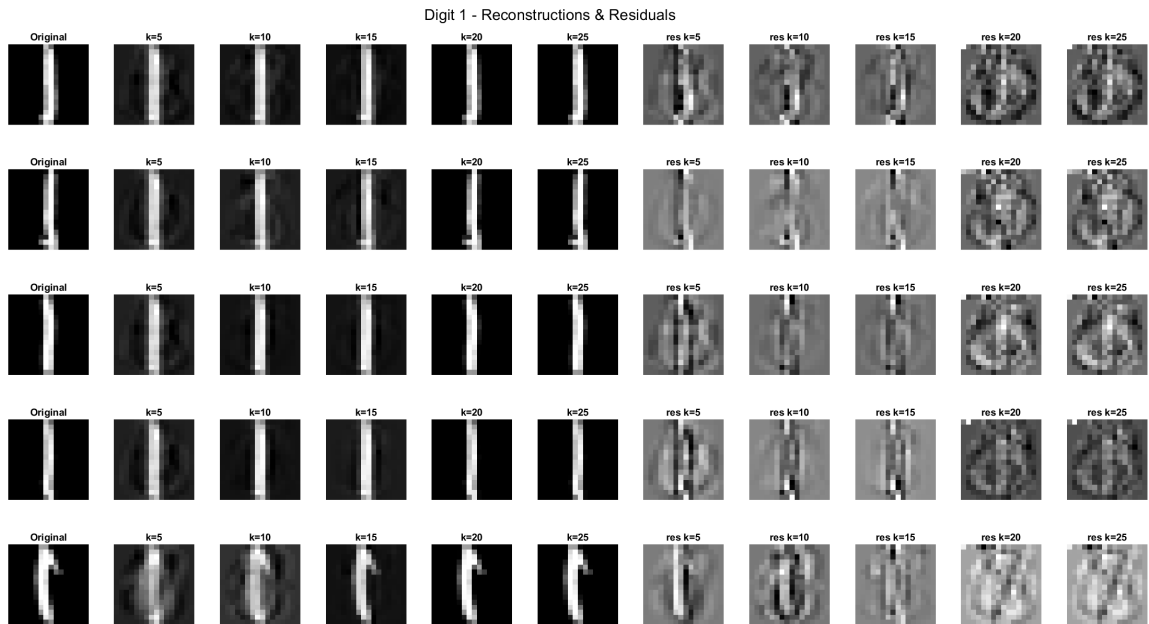


Figure 8. Digit 1: Reconstructions and residuals for varying k values.

Explanation: As k increases, the reconstructed images of digit 1 become noticeably sharper. For $k = 5$ and $k = 10$, the digits appear blurry, especially around the edges, but the overall structure is still recognizable. Starting from $k = 15$, reconstructions resemble the original digits closely, indicating the principal components effectively capture the essential features. The residual images become less pronounced with larger k , confirming that more components lead to better approximation and noise suppression.

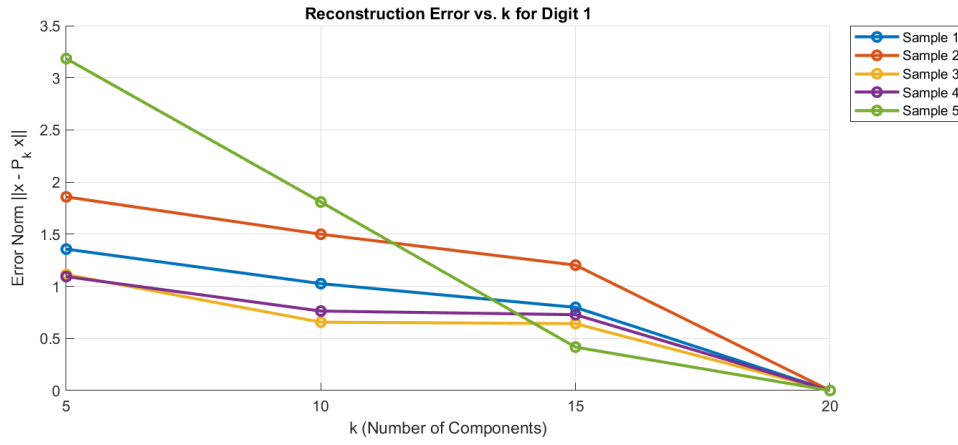


Figure 9. Reconstruction error norm vs. k for Digit 1.

Explanation: The reconstruction error consistently decreases as the number of principal components k increases. This indicates that using more components allows for more accurate approximations. While the rate of improvement slows beyond $k = 15$, the overall downward trend confirms that higher-dimensional subspaces better capture the structure of the digit images, leading to lower residual error norms.



Figure 10. Digit 3: Reconstructions and residuals for varying k values.

Explanation: The reconstructed images of digit 3 are noticeably blurred for low values of k , especially at $k = 5$ and $k = 10$. However, as k increases to 20 and 25, the digits become significantly clearer and visually accurate. The corresponding residuals shrink in intensity, indicating that higher principal components are capturing most of the image structure. This supports the effectiveness of PCA in preserving detail when enough components are retained.

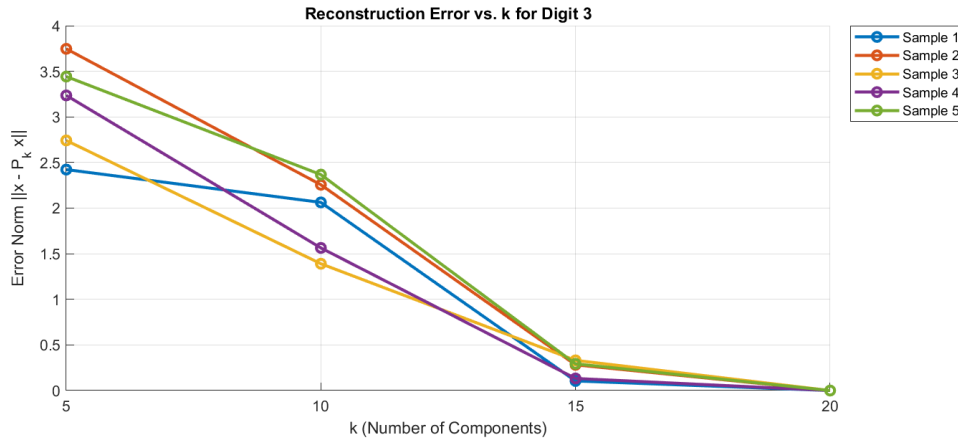


Figure 11. Reconstruction error norm vs. k for Digit 3.

Explanation: The error curves for all five samples decrease sharply as k increases from 5 to 15, showing rapid improvement in reconstruction quality. Beyond $k = 20$, the curves flatten out, indicating that most of the useful variance is already captured and that additional components offer minimal gains.

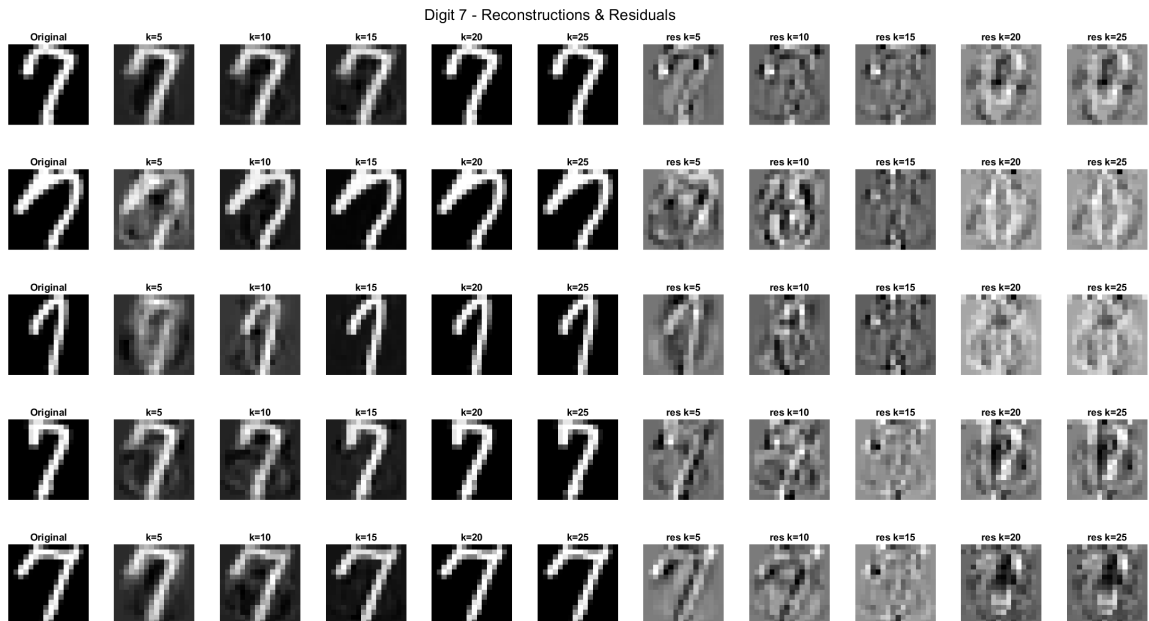


Figure 12. Digit 7: Reconstructions and residuals for varying k values.

Explanation: Even at $k = 5$, the general shape of digit 7 is still clear. As k increases, the images become sharper and more detailed. By $k = 25$, the residuals are very small, showing that the reconstruction is very accurate.

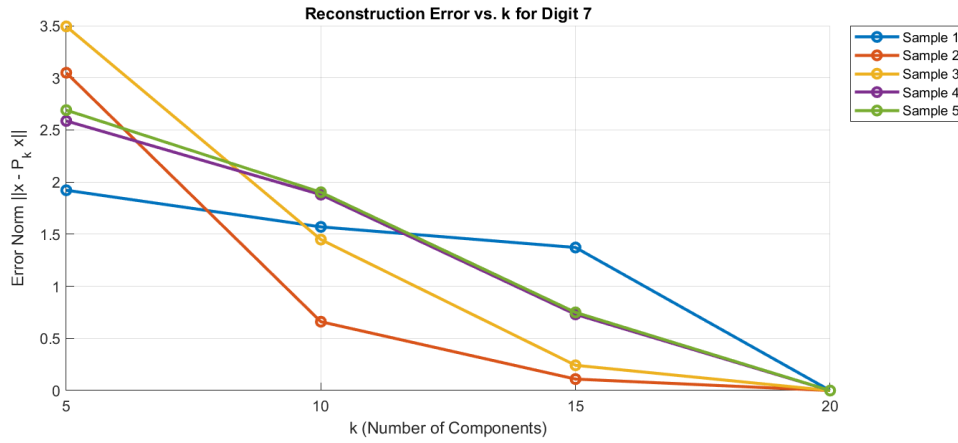


Figure 13. Reconstruction error norm vs. k for Digit 7.

Explanation: The reconstruction error gets smaller as k increases, showing better approximations. Around $k = 20$, the error levels off, meaning more components after this point add little improvement.

3. Exercise 3: PCA-Based Cluster Analysis of Iris Data

3.1. Problem Description

Download from Blackboard the data file `IrisData.mat`. The matrix X consists of 150 vectors, each one having four components. The data correspond to measurements of certain dimensions in three species of flowers, *Iris setosa*, *Iris versicolor*, and *Iris virginica*, and the components of the data vectors have the following attributes:

- sepal length in cm,
- sepal width in cm,
- petal length in cm,
- petal width in cm.

By using the PCA, investigate if the dataset suggests the presence of clusters that would make it possible to separate the three species from each other. (Later on, the flower species corresponding to each data point will be made available.)

3.2. Methodology

The dataset `IrisData.mat` contains 150 samples of flowers, each described by four measurements: sepal length, sepal width, petal length, and petal width. The goal is to use Principal Component Analysis (PCA) to investigate whether the structure of the data suggests natural clusters corresponding to the three species: *Iris setosa*, *Iris versicolor*, and *Iris virginica*.

To begin, the data matrix X was centered by subtracting the mean of each feature, ensuring that the principal components capture variance rather than location. PCA was performed using Singular Value Decomposition (SVD), which allows for an efficient computation of the principal directions and variances. The samples were then projected into two- and three-dimensional principal component (PC) spaces to visualize the data.

```

1 [U, D, V] = svd(Xc, 'econ');
2 PC2 = V(:, 1:2);
3 PC3 = V(:, 1:3);

```

Code 2. Projecting Iris data to PC space

In the 2D PCA projection (PC1 vs. PC2), points were colored based on their true species labels for visual clarity. A separate 3D PCA projection (PC1 vs. PC2 vs. PC3) was also generated. These projections help evaluate whether clusters appear naturally in the reduced space, potentially allowing the three species to be visually separated based on principal component scores.

3.3. Figures and Results

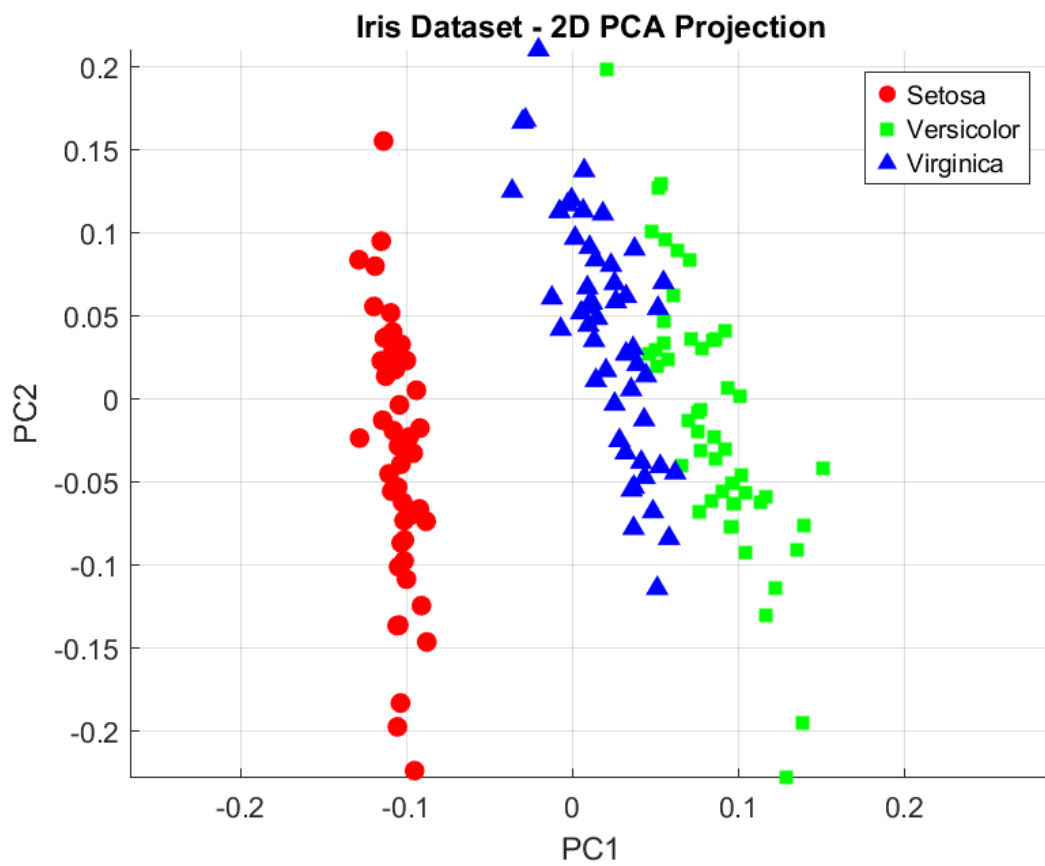


Figure 14. 2D PCA projection. Setosa is clearly separated; Versicolor and Virginica partially overlap.

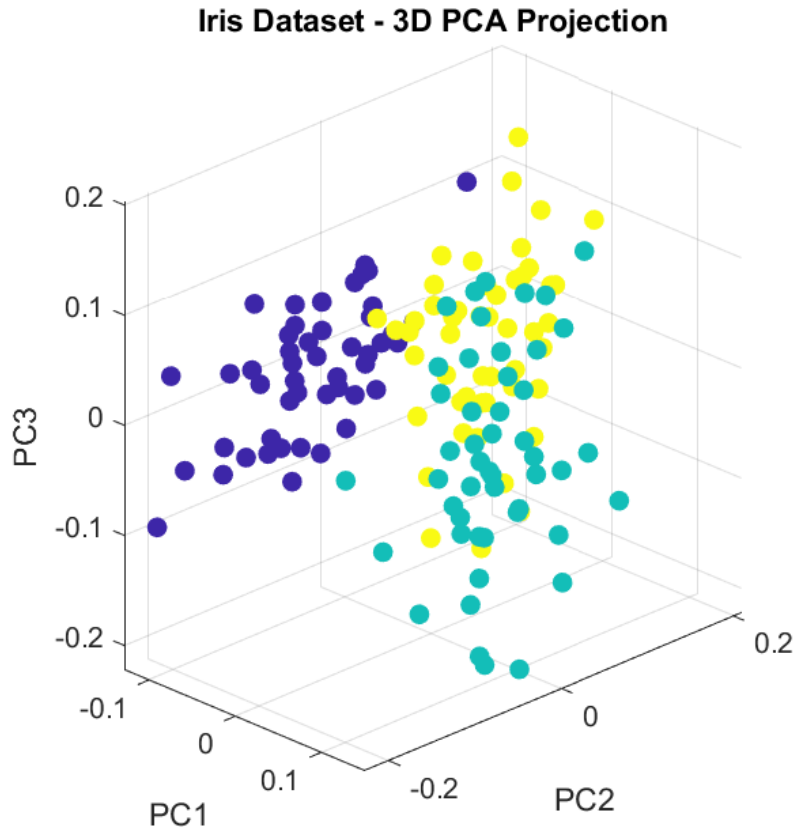


Figure 15. 3D PCA projection. Better visual separation between all three classes.

3.4. Interpretation

The PCA projections reveal a strong underlying cluster structure in the Iris dataset. In the 2D PCA plot, *Iris setosa* forms a well-separated group, indicating it is easily distinguishable from the other species. However, *Iris versicolor* and *Iris virginica* partially overlap, suggesting that two principal components may not be sufficient for perfect separation. The 3D PCA projection improves class separation, particularly between *Versicolor* and *Virginica*, indicating that three components capture more of the variance and help distinguish the species more clearly.

4. Appendix: Full Code Listings

```

1 % visualizing raw data
2 load ModelReductionData
3 for i = 1:5
4     for j = i+1:6
5         figure; % Open a new figure for each plot
6         plot(X(i,:), X(j,:), 'k.', 'MarkerSize', 7);
7         axis equal;
8         set(gca, 'FontSize', 20);
9         xlabel(['Component ', num2str(i)]);
10        ylabel(['Component ', num2str(j)]);
11        title(['Scatter Plot: Component ', num2str(i), ' vs ', num2str(j)]);
12
13        % Generate a filename like "scatter_1_vs_2.png"
14        filename = ['scatter_', num2str(i), '_vs_', num2str(j), '.png'];
15
16        % Save the figure as PNG
17        saveas(gcf, filename);

```

```

18
19     % close the figure to avoid clutter
20     close(gcf);
21 end
22 end
23
24
25 % center the data
26
27 [~, p] = size(X);
28 xbar = 1/p * sum(X,2);
29 Xc = X - xbar * ones(1,p);
30
31 % compute SVD
32
33 [U, D, V] = svd(Xc);
34 sigma = diag(D); % Singular values
35
36 % --- Plot singular values ---
37 figure;
38 plot(sigma, 'bo-', 'LineWidth', 2);
39 xlabel('Component');
40 ylabel('Singular Value');
41 title('Singular Values of Centered Data');
42 grid on;
43 saveas(gcf, 'singular_values.png');
44
45
46 % Compute PCA scores: 4000 x 3
47 PC3 = V(:,1:3) * D(1:3,1:3);
48
49 % PC1 vs PC2
50 figure;
51 plot(PC3(:,1), PC3(:,2), 'k.', 'MarkerSize', 10);
52 xlabel('PC 1'); ylabel('PC 2');
53 title('PCA: PC1 vs PC2');
54 axis equal; grid on;
55 saveas(gcf, 'PCA_PC1_vs_PC2.png');
56
57 % PC1 vs PC3
58 figure;
59 plot(PC3(:,1), PC3(:,3), 'k.', 'MarkerSize', 10);
60 xlabel('PC 1'); ylabel('PC 3');
61 title('PCA: PC1 vs PC3');
62 axis equal; grid on;
63 saveas(gcf, 'PCA_PC1_vs_PC3.png');
64
65 % PC2 vs PC3
66 figure;
67 plot(PC3(:,2), PC3(:,3), 'k.', 'MarkerSize', 10);
68 xlabel('PC 2'); ylabel('PC 3');
69 title('PCA: PC2 vs PC3');
70 axis equal; grid on;

```

```
71 saveas(gcf, 'PCA_PC2_vs_PC3.png');
```

Code 3. Exercise 1 Full Code

```
1 % Load dataset
2 load('HandwrittenDigits.mat'); % X (256x1707), I (1x1707)
3
4 % Parameters
5 target_digits = [0, 1, 3, 7];
6 samples_per_digit = 5;
7 k_values = [5, 10, 15, 20, 25];
8 image_dim = 16;
9
10 % Step 1: Select 5 samples per digit
11 selected_indices = [];
12 digit_labels = [];
13 for d = target_digits
14     idx = find(I == d);
15     selected = idx(1:samples_per_digit);
16     selected_indices = [selected_indices, selected];
17     digit_labels = [digit_labels, repmat(d, 1, samples_per_digit)];
18 end
19
20 % Step 2: Extract subset
21 X_sel = X(:, selected_indices); % 256 x 20
22 [p, n] = size(X_sel); % p = 256, n = 20
23 xbar = mean(X_sel, 2); % mean image
24 Xc = X_sel - xbar * ones(1, n); % center data
25
26 % Step 3: PCA via SVD
27 [U, D, V] = svd(Xc, 'econ');
28 max_k = min(25, size(U, 2));
29 k_vals = k_values(k_values <= max_k); % for error plots
30
31 % Step 4: Plot each digit in its own figure (sample-row layout)
32 for d = 1:length(target_digits)
33     digit = target_digits(d);
34     indices = find(digit_labels == digit); % 5 samples
35
36     figure('Name', ['Digit ' num2str(digit)], 'Position', [100 100 1800 900]);
37
38     for s = 1:samples_per_digit
39         col = indices(s);
40
41         % ---- Plot Original Image ----
42         subplot(samples_per_digit, 11, (s - 1) * 11 + 1);
43         imshow(reshape(X_sel(:, col), [image_dim, image_dim]), []);
44         axis off; axis image;
45         title('Original', 'FontSize', 9);
46
47         for ki = 1:length(k_values)
48             k = k_values(ki);
49             max_k = min(k, size(U, 2));
```

```

50     Uk = U(:, 1:max_k);
51     z = Uk' * Xc(:, col);
52     xk = Uk * z + xbar;
53     residual = X_sel(:, col) - xk;
54
55     % ---- Plot Reconstruction ----
56     subplot(samples_per_digit, 11, (s - 1) * 11 + 1 + ki);
57     imshow(reshape(xk, [image_dim, image_dim]), []);
58     axis off; axis image;
59     title(['k=' num2str(k)], 'FontSize', 9);
60
61     % ---- Plot Residual ----
62     subplot(samples_per_digit, 11, (s - 1) * 11 + 1 + ki + length(k_values));
63     imshow(reshape(residual, [image_dim, image_dim]), []);
64     axis off; axis image;
65     title(['res k=' num2str(k)], 'FontSize', 9);
66 end
67 end
68
69 sgtitle(['Digit ' num2str(digit) ' - Reconstructions & Residuals '], 'FontSize', 14);
70 saveas(gcf, ['digit_' num2str(digit) '_layout_by_sample.png']);
71
72 % -----
73 % Plot error norm vs. k for each sample
74 % -----
75 figure('Name', ['Error Norms - Digit ' num2str(digit)], 'Position', [100 100 1000 400]);
76 hold on;
77
78 for s = 1:samples_per_digit
79     col = indices(s);
80     errors = zeros(1, length(k_vals));
81
82     for ki = 1:length(k_vals)
83         k = k_vals(ki);
84         Uk = U(:, 1:k);
85         z = Uk' * Xc(:, col);
86         xk = Uk * z + xbar;
87         residual = X_sel(:, col) - xk;
88         errors(ki) = norm(residual);
89     end
90
91     plot(k_vals, errors, '-o', 'LineWidth', 2, 'DisplayName', ['Sample ' num2str(s)]);
92 end
93
94 xlabel('k (Number of Components)');
95 ylabel('Error Norm ||x - P_k x||');
96 title(['Reconstruction Error vs. k for Digit ' num2str(digit)]);
97 legend('Location', 'northeastoutside');
98 grid on;
99
100 saveas(gcf, ['digit_' num2str(digit) '_error_plot.png']);
101 end

```


Code 4. Exercise 2 Full Code

```

1
2
3 % Load data
4 load('IrisDataAnnotated.mat');
5
6 % Ensure X is 4x150 (features x samples), I is 150x1
7 [p, n] = size(X); % Should be 4x150
8 if p ~= 4
9     X = X'; % Fix if needed
10 end
11
12 % Center the data (column-wise)
13 xbar = mean(X, 2); % 4x1
14 Xc = X - xbar * ones(1, n); % Centered: 4x150
15
16 % Perform PCA via SVD
17 [U, D, V] = svd(Xc, 'econ'); % U: 4x4, D: 4x4, V: 150x4
18
19 % Projection of points into PC space (rows = samples)
20 PC2 = V(:,1:2); % 150x2
21 PC3 = V(:,1:3); % 150x3
22
23 % Plot 2D PCA projection
24 figure('Name', 'PCA - 2D');
25 hold on;
26 colors = ['r', 'g', 'b'];
27 markers = ['o', 's', '^'];
28
29 for c = 1:3
30     idx = I == c;
31     scatter(PC2(idx,1), PC2(idx,2), 50, colors(c), markers(c), 'filled');
32 end
33
34 xlabel('PC1'); ylabel('PC2');
35 title('Iris Dataset - 2D PCA Projection');
36 legend('Setosa', 'Versicolor', 'Virginica');
37 axis equal; grid on;
38 hold off;
39
40 xlabel('PC1'); ylabel('PC2');
41 title('Iris Dataset - 2D PCA Projection');
42 axis equal; grid on;
43 legend('Setosa', 'Versicolor', 'Virginica');
44 saveas(gcf, 'iris_pca_2d.png');
45
46 % Plot 3D PCA projection
47 figure('Name', 'PCA - 3D');
48 scatter3(PC3(:,1), PC3(:,2), PC3(:,3), 50, I, 'filled');
49 xlabel('PC1'); ylabel('PC2'); zlabel('PC3');

```

```
50 title('Iris Dataset - 3D PCA Projection');  
51 grid on; axis equal;  
52 view(45, 25);  
53 saveas(gcf, 'iris_pca_3d.png');
```

Code 5. Exercise 3 Full Code